User Guide: Deploying a Local Agentic Workflow for ARPES Data Processing

Version of 30 October 2026. Written from a working deployment: Claude Sonnet 5.5 (medium effort) driving the open-source ARPES viewer v1.2.2 (PyQt5 / pyqtgraph) on a Windows workstation with two monitors. All paths below are placeholders; replace them with your own.

1. What this gives you

You talk to Claude in one chat window (Chinese or English). Claude plans the processing, runs it **inside the ARPES viewer on your own computer**, shows you the result in the viewer on a second monitor, and asks you whenever a physical parameter is uncertain. The report is written in English with sample information removed.

Hard guarantees of this design

- Raw data and source code stay on your computer. The model sees only text output, screenshots and finished figures that you allow.
- You do not need to click or type in the viewer: Claude drives it through local scripts.
- Every step uses the viewer's own functions unless Claude states otherwise, so every number can be reproduced by hand.

2. Architecture in one picture

```
Chat window (you)  <---->  Claude (cloud model)
                                |  shell + file tools of the Claude desktop app
                                v
Your computer (Windows)
  ├─ connected folder: <REPO>\_agent\
  │    ├─ queue\        jobs for the batch runner         (*.py)
  │    ├─ viewer_cmd\   commands for the live viewer       (*.py)
  │    ├─ out\          text logs of every job/command
  │    ├─ shots\        screenshots grabbed by the viewer
  │    └─ figures\      figures written by the viewer
  ├─ agent_runner.pyw   polls queue\ -> runs jobs with python.exe (offscreen Qt)
  └─ viewer_live.py     the real viewer window on screen 2; polls viewer_cmd\ and
                        exec()s each command inside the running GUI process
```

Why two mechanisms? The *runner* is for heavy or headless work (no window needed). The *live viewer* is for anything you want to look at: Claude executes commands in the same process that owns the window, so dialogs, the main list and figures are the real thing, and screenshots are taken with `widget.grab()` from inside that process (no screen capture, no desktop control, nothing extra to authorise).

3. Prerequisites

Need	Notes
Claude desktop app, signed in	Linking the conversation to this computer gives Claude a shell restricted to the folders you connect

Need	Notes
Windows Python 3.10+ (3.14 was used)	With `numpy`, `scipy`, `h5py`, `PyQt5`, `pyqtgraph`, `matplotlib`
The ARPES viewer source	Clone from https://github.com/DingPei1995/ARPES-viewer into `<REPO>`
Two monitors (recommended)	Viewer on screen 2, chat on screen 1
A model setting	Sonnet-class model, medium effort is enough for routine processing

Install packages once (your own terminal):

```
python -m pip install numpy scipy h5py PyQt5 pyqtgraph matplotlib
```

4. One-time setup

4.1 Link the computer and connect folders

1. Open the task in the Claude desktop app on the workstation and choose **Link to this computer**.
2. Grant access to `<REPO>` (the viewer repository, where `_agent\` will live) and to the data folder `<DATA_DIR>`. Grant the **data folder read-only** if the app offers it; Claude never needs to write there.
3. Deleting files is off by default. When Claude needs to delete something it asks permission for the connected folder; approve once, and only for the folders that need it.

4.2 Create the working folders

```
<REPO>\_agent\queue
<REPO>\_agent\out
<REPO>\_agent\done
<REPO>\_agent\viewer_cmd
<REPO>\_agent\viewer_done
<REPO>\_agent\shots
<REPO>\_agent\figures
```

Add `_agent/` to the repository's `.gitignore` so that nothing from the session is ever committed.

4.3 The local job runner (`agent_runner.pyw`)

Pure standard library, no network access. Start it by double-clicking (associate `.pyw` with your Python once). To stop it, create a file called `STOP` next to it.

```
        import os, sys, time, subprocess, shutil, datetime
        HERE = os.path.dirname(os.path.abspath(__file__))
        Q, OUT, DONE = (os.path.join(HERE, d) for d in ("queue", "out", "done"))
        py = sys.executable
        cand = os.path.join(os.path.dirname(py), "python.exe")
        if py.lower().endswith("pythonw.exe") and os.path.exists(cand):
            py = cand
        def beat(msg="alive"):
            with open(os.path.join(HERE, "heartbeat.txt"), "w") as f:
                f.write(f"{datetime.datetime.now().isoformat()} {msg} | {py}\n")
        stop = os.path.join(HERE, "STOP")
        if os.path.exists(stop): os.remove(stop)
        beat("started")
        while not os.path.exists(stop):
            beat()
            for n in sorted(os.listdir(Q)):
                if not n.endswith(".py"): continue
                src = os.path.join(Q, n); time.sleep(0.3)
                log = os.path.join(OUT, n[:-3] + ".log")
                env = dict(os.environ, QT_QPA_PLATFORM=os.environ.get("QT_QPA_PLATFORM",
"offscreen"),
                           PYTHONIOENCODING="utf-8", HF_HUB_OFFLINE="1")
                with open(log, "w", encoding="utf-8") as lf:
                    t0 = time.time()
                    try:
                        r = subprocess.run([py, src], stdout=lf, stderr=subprocess.STDOUT,
cwd=HERE,
                                           env=env, creationflags=0x08000000, timeout=3600)
                        lf.write(f"\n[exit {r.returncode}, {time.time()-t0:.1f}s]\n")
                    except Exception as e:
                        lf.write(f"\n[runner error] {e!r}\n")
                shutil.move(src, os.path.join(DONE, n))
            time.sleep(1)
        beat("stopped")
```

`heartbeat.txt` lets Claude check that the runner is alive. Each job writes a log in `out\` and is then moved to `done\`.

4.4 The live viewer with a command poller (`viewer_live.py`)

Launch the real viewer, move its window to the second screen, list your data files, and poll `viewer_cmd\` every 0.7 s. Each command file is executed in the GUI thread with the application module available as `A`:

```
        import os, sys, glob, time, traceback, io, contextlib
        ROOT = r"<REPO>"; DATA = r"<DATA_DIR>"
        HERE = os.path.dirname(os.path.abspath(__file__))
        sys.path.insert(0, ROOT); os.environ["QT_QPA_PLATFORM"] = "windows"
        import arpes_viewer.app as A
        from PyQt5 import QtCore, QtWidgets

        def setup():
            w = A.win
            scr = QtWidgets.QApplication.screens()
            if len(scr) > 1:                                    # window to the second screen
                g = [s for s in scr if s != QtWidgets.QApplication.primaryScreen()]
[0].geometry()
                w.move(g.x() + 40, g.y() + 40)
            A.add_files(sorted(glob.glob(os.path.join(DATA, "*.txt"))), background=False)
            w.show(); w.raise_()
            w.grab().save(os.path.join(HERE, "shots", "step01_main_panel_loaded.png"))

        def poll():
            qd, dd = os.path.join(HERE, "viewer_cmd"), os.path.join(HERE, "viewer_done")
            for n in sorted(os.listdir(qd)):
                if not n.endswith(".py"): continue
                src = os.path.join(qd, n); time.sleep(0.2)
                buf = io.StringIO()
                with contextlib.redirect_stdout(buf), contextlib.redirect_stderr(buf):
                    try:
                        exec(compile(open(src, encoding="utf-8").read(), src, "exec"),
                             {"A": A, "HERE": HERE, "DATA": DATA, "QtCore": QtCore,
"QtWidgets": QtWidgets})
                    except Exception:
                        traceback.print_exc()
                open(os.path.join(HERE, "out", "cmd_" + n[:-3] + ".log"), "w", encoding="utf-
8").write(buf.getvalue())
                os.replace(src, os.path.join(dd, n))

        _orig = A.offer_recovery
        def patched():
            QtCore.QTimer.singleShot(300, setup)
            t = QtCore.QTimer(A.win); t.timeout.connect(poll); t.start(700); A._poll_timer =
t
        A.offer_recovery = patched
        sys.exit(A.main())
```

Start it detached (so it survives the launcher) with `QT_QPA_PLATFORM=windows`:

```
        import os, sys, subprocess
        py = sys.executable.replace("pythonw.exe", "python.exe")
        env = dict(os.environ, QT_QPA_PLATFORM="windows", PYTHONIOENCODING="utf-8")
        subprocess.Popen([py, r"<REPO>\_agent\viewer_live.py"], env=env, cwd=r"
<REPO>\_agent",
                         creationflags=0x00000008 | 0x00000200 | 0x08000000)   # detached,
new group, no console
```

Rename rows to the file or folder names so that your main list is easy to audit.

4.5 First check

Ask Claude to confirm that (1) `heartbeat.txt` is recent and (2) a screenshot of the main panel in `shots\` shows all datasets listed. You should see the viewer on screen 2 with the files loaded. Nothing has been copied or converted yet.

5. Tell Claude the rules (paste at the start of a session)

```
1. All processing is local. Never upload raw data or code anywhere. Screenshots and
   finished figures may be shown to you.
2. Use the ARPES viewer's own functions by default. If you write your own code, say so.
3. If a parameter is uncertain (Gamma point, windows, thresholds), open the data and
   ask me before proceeding.
4. Real runs: keep only the original data and the final-step result in the main list;
   remove intermediates and their autosave files. Do not remove anything I did not ask for.
5. Convert maps to k-space with at least 100 points on the scan axis; figures default to
   Smooth ticked, blur 0.
6. Screenshot every key step for the report.
7. Reports in English; hide sample name and lattice constants (use a, b, c); polarisation
   labels (LH, LV, CR, CL) may stay.
```

6. Recipes (viewer v1.2.2 internal API)

These are the calls used in the session, shown as command files for `viewer_cmd\`. They use internal functions of this version of the viewer and may change between releases; the dialogs themselves are the stable public interface.

Normalise every cut by its total intensity (select rows, then use the program's dialog):

```
        L = A.ui.FilePathListWidget
        dlg = A.open_data_operations("normalise")
        dlg.norm_preset.setCurrentIndex(dlg.norm_preset.count() - 1)       # custom
        for c in dlg.norm_checks: c.setChecked(True)                         # sum over both
axes
        res = dlg.apply(); dlg.close()
```

k conversion of a cut — remember the *Advanced* switch, otherwise the Gamma deflector angle is ignored:

```
        d = A.open_k_conversion("cut")
        d.advanced.setChecked(True)
        # set cut deflector angle, Gamma deflector angle, Gamma slit angle in the dialog's
fields
        d.to_list.click()
```

Fermi edge fit and energy offset on a converted dataset:

```
        fd = A.open_fermi_fit()
        fd.estimate(); fd.run_fit()
        fd.offset_energy_axis(whole_map=True)    # use whole_map=False for single cuts
```

Other building blocks: `A.open_brillouin_zone()`, `A.open_slice_series()`, `A.figure_window(panels)` (then `win.write(path, "png")` and `win.save_file(path_afig)`), `A.map_contour(key).run_arbitrary_cut(points, name=..., open_window=False)` for a cut along any line of a map, `A._remove_keys(keys)` to remove list rows (it also clears their autosave files), and `arpes_viewer.tools.spectra.peaks.fit_line(...)` for two-peak MDC fits.

Screenshot of any window or dialog (inside the live viewer; no screen capture needed):

```
        dlg.grab().save(HERE + r"\shots\stepNN_name.png")
```

Checking the result. Ask Claude to write each command's printed output to `out\` (done automatically above), to name every new row clearly (e.g. `<file>_s_nor [k] [E-Ef]`), and to summarise what changed in the main list after each step so that you can audit it on screen 2.

7. A typical session

1. Load and rename the data; you check the main list.
2. Normalise each map plane (cut-wise) and compare to the raw intensity profile.
3. Check Fermi-level flatness; apply the correction only if the variation exceeds the resolution.
4. Pick Gamma on a contour (Claude opens the data and asks you); convert maps to k space.
5. Fit the Fermi edge, offset the energy axis, remove intermediate rows.
6. Overlay the Brillouin zone and make iso-energy figures; you tune the levels and gamma.
7. Convert, calibrate and normalise cuts; build comparison figures (LH/LV, LD, CR/CL, CD).
8. Quantify (e.g. MDC fits vs temperature) and draw the summary figure.
9. Ask for the English report; sample information is hidden automatically.

8. Pitfalls we hit (and how to avoid them)

Problem	Cause	Fix
Gamma deflector angle silently ignored	Option only active with *Advanced: Gamma off this cut* ticked	Always tick it; verify the `kcut.*` metadata of the result
Fermi-edge fit unstable on warm data	Edge broadened, low counts at small scan angles	Use the full slit window, hold the temperature, check against resolution before flattening
Windows with the wrong geometry after a dialog	Modal dialogs block the command poller	Fill the dialog from a timer and accept, or close stray windows before the next command
A large duplicate of the data was written to disk	Agent tried to "bundle" the data	Forbid copies; list the original files directly (the live viewer does)
Rows removed that you wanted to keep	Ambiguous instruction	Spell out "remove only X"; ask Claude to list the rows before and after
Windows app tiers	File Explorer is click-only; `pythonw` cannot be controlled	Double-click launchers yourself once; everything else runs through the poller
No window available	The batch runner runs offscreen	Use the live viewer for anything visual

9. Privacy checklist before sharing a report

- Replace the sample name and any lattice constants by generic symbols (a, b, c).
- Crop or blank the folder-path field in main-panel screenshots (it contains facility paths).
- Check figure titles, file names and window titles for sample or proposal identifiers.
- Do not attach raw data, source files or the `_agent\` scripts.

10. Cost and speed control

- A processing session is dominated by re-reading the conversation (cache reads), not by new text. Start a new session per processing stage and keep screenshots few and small.
- Read the viewer's source locally with targeted searches rather than fetching a whole repository through a connector.
- Reuse scripts: ask Claude to save each successful command in `viewer_done\` and re-run it for the next dataset instead of re-explaining the step.
- Record the usage before and after a session (account usage page) in addition to the token counts in the experiment update.

11. Teardown

1. Create `<REPO>\_agent\STOP` to stop the runner and close the viewer.
2. Move the report and figures you want to keep to a permanent folder, then delete `<REPO>\_agent\` (ask Claude to request the delete permission, or delete it yourself).
3. Remove the `_agent/` line from `.gitignore` if you no longer need it, and revoke the folder grants in the Claude desktop app.